

The Deterministic Agent — System Design Document

Version: 1.0

Author: Nishant

Date: March 2026

Status: Draft

1. Executive Summary

The Deterministic Agent is a system that wraps a coding agent with a **decision-tracing and path-locking layer**, enabling reproducibility, exploration, and comparison of the multiple valid solution paths an AI agent can take when solving a programming task.

The core insight: when an LLM-powered coding agent tackles a moderately complex task, the problem isn't that it fails — it's that it succeeds *differently* every time. The system captures the structured decision trace of each run, allows exact replay via path locking, and enables users to fork at any decision point to explore counterfactual paths.

The reference task (stock anomaly detection pipeline) is intentionally chosen to surface ~7+ genuine branching points before a single line of code is written.

2. Problem Statement

2.1 Core Challenge

Given a coding agent and a moderately complex task, each run produces a functionally correct but structurally different solution. These differences arise not from randomness (temperature) but from **legitimate decision-path divergence** — the agent makes choices at branching points that cascade into fundamentally different solution architectures.

2.2 What We're Building

A system that:

1. **Captures** a structured decision trace for every agent run
2. **Replays** a previous run deterministically via path locking
3. **Forks** from any decision point to explore alternative paths
4. **Compares** runs to identify which branching points drive the most variance

2.3 What We're NOT Building

- A general-purpose agent framework (we're building a specific capability layer)
- A prompt-optimization system (decisions are captured, not tuned)

- A code-quality evaluator (all paths are assumed valid if tests pass)

3. Functional Requirements

3.1 FR-1: Coding Agent (The Runner)

ID	Requirement	Priority
FR-1.1	Agent accepts a natural-language task description and produces a working codebase	P0
FR-1.2	Agent uses an LLM to plan, write code, execute it, evaluate output, and iterate	P0
FR-1.3	Agent pulls real data via yfinance (no mocked datasets)	P0
FR-1.4	Agent handles errors by reasoning about failures and attempting recovery strategies	P0
FR-1.5	Agent makes genuine decisions at each branching point (ticker, timeframe, method, framework)	P0
FR-1.6	Agent produces a complete deliverable: anomaly detection + report + API endpoint	P0
FR-1.7	Agent supports configurable LLM backend (OpenAI, Anthropic, local)	P1

3.2 FR-2: Decision Trace

ID	Requirement	Priority
FR-2.1	Every run captures a structured decision trace as Pydantic models (not free text)	P0
FR-2.2	Each decision point records: alternatives considered, choice made, reasoning, confidence	P0
FR-2.3	Trace captures code written at each step	P0
FR-2.4	Trace captures errors hit and recovery strategies attempted	P0
FR-2.5	Trace captures execution results (stdout, stderr, exit codes)	P0

ID	Requirement	Priority
FR-2.6	Trace includes timing metadata (wall clock, LLM latency per step)	P1
FR-2.7	Traces are serializable to JSON and loadable from disk	P0
FR-2.8	Trace includes a DAG of decision dependencies (which decisions enabled/constrained others)	P2

3.3 FR-3: Path Locking

ID	Requirement	Priority
FR-3.1	Given a previous run's decision trace, a new run follows the same decisions	P0
FR-3.2	Locked decisions are injected into the LLM prompt context, not post-hoc filtered	P0
FR-3.3	The system detects when a locked path diverges (e.g., an API changed) and reports it	P0
FR-3.4	Partial locking is supported: lock decisions 1-3, let 4+ run freely	P1
FR-3.5	Path locking works across LLM provider changes (lock from GPT-4 run, replay on Claude)	P2

3.4 FR-4: Fork & Explore

ID	Requirement	Priority
FR-4.1	User can select any decision point and override the choice	P1
FR-4.2	Upstream decisions are frozen; downstream decisions run freely from the fork point	P1
FR-4.3	Forked runs produce comparable output artifacts for side-by-side comparison	P1
FR-4.4	Fork history is tracked (which run was forked, at which decision point, with what override)	P1

3.5 FR-5: Path Analysis

ID	Requirement	Priority
FR-5.1	Across N runs, visualize the decision tree showing all paths taken	P1
FR-5.2	Identify which branching points have the highest variance across runs	P1
FR-5.3	Correlate path choices with outcome metrics (code size, execution time, test pass rate)	P2

3.6 FR-6: CLI Interface

ID	Requirement	Priority
FR-6.1	<code>agent run <task></code> — Execute a new agent run	P0
FR-6.2	<code>agent replay <run-id></code> — Replay a locked path	P0
FR-6.3	<code>agent trace <run-id></code> — View decision trace	P0
FR-6.4	<code>agent list</code> — List all recorded runs	P0
FR-6.5	<code>agent fork <run-id> --decision <id> --choice <value></code> — Fork from a decision point	P1
FR-6.6	<code>agent compare <run-id-1> <run-id-2></code> — Side-by-side comparison	P1
FR-6.7	<code>agent analyze</code> — Cross-run path analysis	P2

3.7 FR-7: API Interface (Stretch)

ID	Requirement	Priority
FR-7.1	<code>POST /runs</code> — Trigger a new run	P2
FR-7.2	<code>GET /runs</code> — List runs	P2
FR-7.3	<code>GET /runs/{id}/trace</code> — Get decision trace	P2

ID	Requirement	Priority
FR-7.4	<code>POST /runs/{id}/replay</code> — Replay a locked path	P2
FR-7.5	<code>POST /runs/{id}/fork</code> — Fork from a decision point	P2
FR-7.6	<code>WS /runs/{id}/stream</code> — Real-time execution streaming	P2

4. Non-Functional Requirements

4.1 Reproducibility

ID	Requirement	Target
NFR-1.1	Path-locked runs must produce functionally identical output	100% decision match when external APIs haven't changed
NFR-1.2	Decision trace must be complete enough to reconstruct the agent's reasoning	Every LLM call and its structured output is captured
NFR-1.3	System must detect and report non-reproducibility causes	Drift report with specific divergence point

4.2 Performance

ID	Requirement	Target
NFR-2.1	Decision trace overhead vs. untraced run	< 5% wall-clock overhead
NFR-2.2	Trace serialization latency	< 100ms per decision point
NFR-2.3	Path locking prompt injection latency	< 50ms per locked decision
NFR-2.4	Full run completion (reference task)	< 10 minutes (dominated by LLM latency)

4.3 Storage

ID	Requirement	Target
NFR-3.1	Decision trace size per run	< 5MB (JSON)
NFR-3.2	Generated code artifacts per run	< 50MB
NFR-3.3	System must support 100+ stored runs	Filesystem-backed, no DB required for MVP

4.4 Reliability

ID	Requirement	Target
NFR-4.1	Agent must handle yfinance API failures gracefully	Retry with backoff, fallback ticker
NFR-4.2	Partial run recovery: if agent crashes mid-run, trace up to crash point is preserved	Incremental trace writes
NFR-4.3	Path locking must fail loudly if a locked decision can't be applied	Raise <code>PathLockDivergenceError</code>

4.5 Extensibility

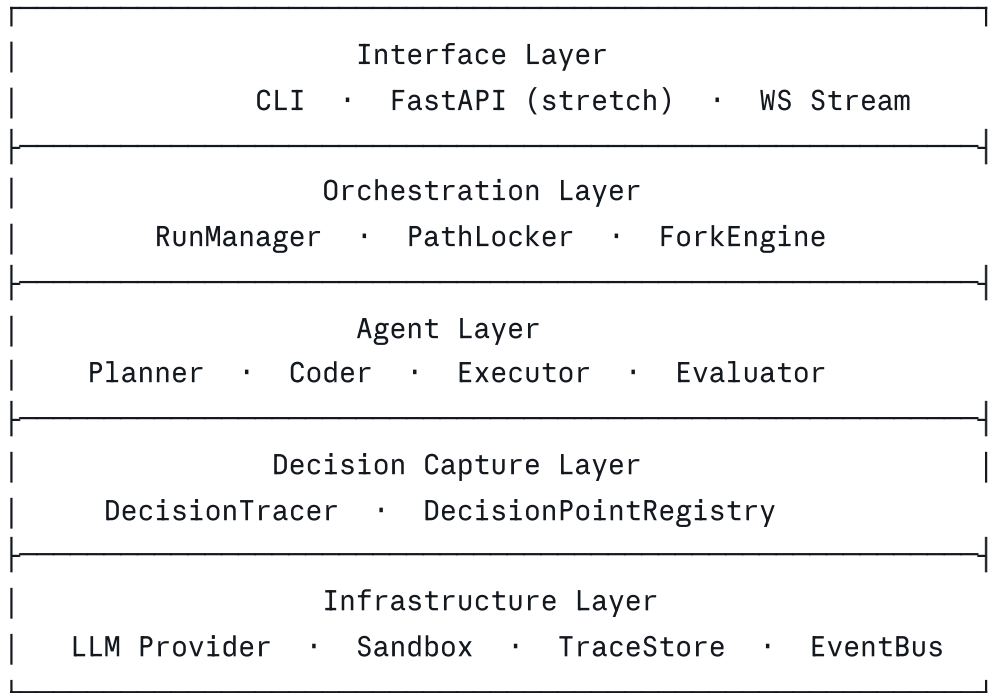
ID	Requirement	Target
NFR-5.1	Adding a new decision-point type requires < 30 lines of code	Plugin-style decision point registry
NFR-5.2	Swapping LLM providers requires changing 1 config value	Provider abstraction layer
NFR-5.3	The reference task (stock anomaly) is replaceable with any other task	Task description is the only input

4.6 Observability

ID	Requirement	Target
NFR-6.1	Every LLM call is logged with prompt, response, latency, token count	Structured logging (JSON)
NFR-6.2	Agent progress is observable in real-time during execution	Event emitter / callback hooks
NFR-6.3	Decision points are emitted as structured events as they occur	SSE or WebSocket for stretch API

5. High-Level Architecture

5.1 Conceptual Layers



5.2 Component Overview

Interface Layer — Entry points for users. CLI is P0; FastAPI + WebSocket streaming is P2.

Orchestration Layer — Coordinates runs, manages path locking injection, handles fork logic. This is the "brain" that decides whether the agent is running freely or following a locked path.

Agent Layer — The actual coding agent. A loop of: Plan → Code → Execute → Evaluate → (iterate or advance). Each phase can produce decision points.

Decision Capture Layer — Intercepts agent decisions and records them as structured Pydantic models. The `DecisionPointRegistry` defines the taxonomy of decision types.

Infrastructure Layer — LLM abstraction, code sandbox for safe execution, persistent trace storage, and an event bus for real-time observability.

6. Detailed Component Design

6.1 Agent Core (The Runner)

The agent operates as a **state machine** with the following phases:

PLANNING → CODING → EXECUTING → EVALUATING → (loop or DONE)

Phase: PLANNING

- Input: Task description + any locked decisions + execution history so far
- Output: A structured plan with explicit decision points
- Key: The planner doesn't just decide — it enumerates alternatives before choosing

Phase: CODING

- Input: Plan + decisions made + prior code (if iterating)
- Output: Code files written to the sandbox
- Key: Code generation is decision-traced (which libraries, which patterns)

Phase: EXECUTING

- Input: Code files in sandbox
- Output: stdout, stderr, exit code, generated artifacts
- Key: Execution happens in an isolated subprocess with timeout

Phase: EVALUATING

- Input: Execution results
- Output: Assessment (success / partial / failure) + next action
- Key: On failure, the evaluator decides recovery strategy (fix code, change approach, change library) — this is itself a traced decision

Agent Loop Pseudocode

```
python
```



```

class AgentRunner:
    def run(self, task: str, locked_trace: Optional[DecisionTrace] = None) -> RunRes
        tracer = DecisionTracer()
        locker = PathLocker(locked_trace) if locked_trace else None

        plan = self.plan(task, tracer, locker)

        for iteration in range(MAX_ITERATIONS):
            code = self.code(plan, tracer, locker)
            result = self.execute(code)
            tracer.record_execution(result)

            assessment = self.evaluate(result, tracer, locker)

            if assessment.status == "success":
                return RunResult(trace=tracer.finalize(), artifacts=result.artifacts)

            if assessment.status == "fatal":
                return RunResult(trace=tracer.finalize(), error=assessment.error)

            # Iterate: assessment contains recovery strategy (a traced decision)
            plan = self.replan(plan, assessment, tracer, locker)

        return RunResult(trace=tracer.finalize(), error="max iterations exceeded")

```

6.2 Decision Tracer

The tracer is injected into every agent phase. It intercepts decision moments and records them.

Decision Point Taxonomy

For the reference task, we identify these decision-point categories:

Category	Example Decision Points	Cardinality
data_selection	Ticker(s), timeframe, data fields, data source	4-6 per run
algorithm_selection	Anomaly detection method, parameters, thresholds	2-4 per run
architecture	API framework, report format, project structure	3-5 per run
library_selection	pandas vs polars, matplotlib vs plotly, etc.	2-4 per run
error_recovery	Fix strategy on failure (retry, change approach, fallback)	0-N per run
optimization	Caching, batching, async, performance choices	0-3 per run

Decision Point Structure (Pydantic)

python

```

class Alternative(BaseModel):
    """One option the agent considered."""
    value: str # e.g., "IQR"
    reasoning: Optional[str] = None # Why this was considered

class DecisionPoint(BaseModel):
    """A single branching point in the agent's execution."""
    id: str # Unique ID, e.g., "dp-001"
    sequence_number: int # Ordinal position in the run
    timestamp: datetime
    category: DecisionCategory # Enum: data_selection, algorithm_selection,
    phase: AgentPhase # Enum: planning, coding, executing, evaluating
    question: str # "Which anomaly detection method?"
    alternatives: list[Alternative] # What was considered
    chosen: str # What was selected
    reasoning: str # Why this choice was made
    confidence: float # 0.0 - 1.0
    parent_decision_id: Optional[str] # Which prior decision constrained this one
    context_snapshot: dict # Relevant state at decision time
    locked: bool = False # Was this decision forced by path locking?

class ExecutionRecord(BaseModel):
    """Record of a code execution attempt."""
    id: str
    code_snapshot: str # Full code at this point
    command: str # What was executed
    stdout: str
    stderr: str
    exit_code: int
    duration_ms: int
    artifacts_produced: list[str] # File paths of outputs

class DecisionTrace(BaseModel):
    """Complete trace of a single agent run."""
    run_id: str
    task_description: str
    started_at: datetime
    completed_at: Optional[datetime]
    status: RunStatus # success, failed, partial
    decisions: list[DecisionPoint]
    executions: list[ExecutionRecord]
    llm_calls: list[LLMCallRecord] # Every prompt/response pair
    metadata: RunMetadata # LLM provider, model, temperature, etc.
    parent_run_id: Optional[str] # If forked, which run was the source
    fork_point: Optional[str] # If forked, which decision was overridden

```

How Decisions Are Captured

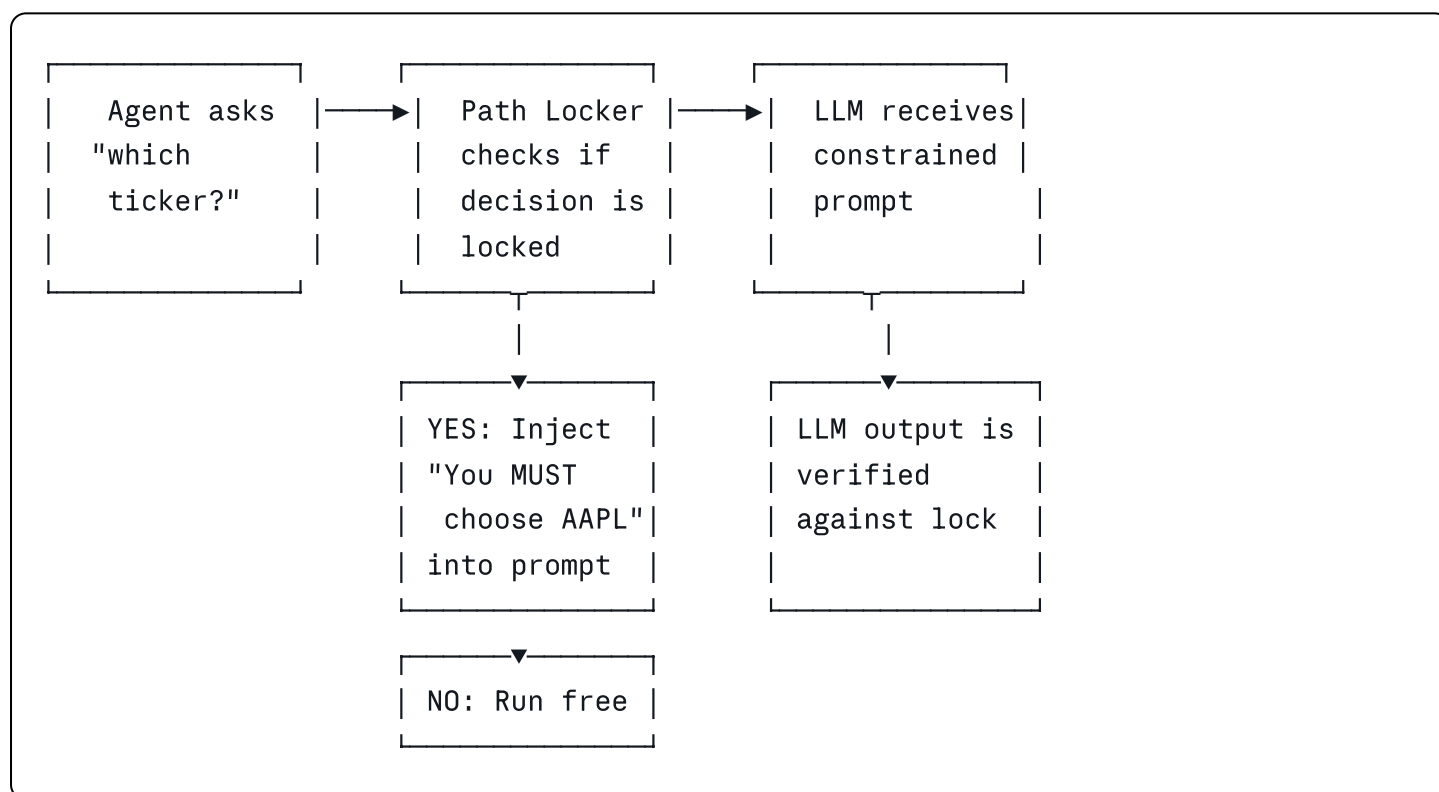
The tracer doesn't passively log — it **structures the LLM's output**. The key mechanism:

1. **Structured Output Prompting:** At every decision point, the LLM is prompted to output a JSON object matching the `DecisionPoint` schema. The prompt explicitly asks for alternatives, reasoning, and confidence.
2. **Schema Enforcement:** The LLM response is parsed into the Pydantic model. If parsing fails, the tracer retries with a correction prompt.
3. **Incremental Persistence:** Each decision point is written to disk as it occurs (append to JSONL), so partial runs are recoverable.

6.3 Path Locker

This is the **core design challenge**. The path locker takes a previous trace and constrains a new run to follow the same decisions.

Locking Strategy: Prompt Injection with Verification



Three-Layer Locking:

1. **Prompt-Level Lock (Primary):** The locked decision is injected into the system prompt as a hard constraint. E.g., *"For the anomaly detection method, you MUST choose IQR. Do not consider other options."*
2. **Output Verification (Secondary):** After the LLM responds, the tracer verifies the chosen value matches the locked value. If it doesn't match (LLM ignored the constraint), the system retries with a stronger prompt or raises a `PathLockViolationError`.

3. **Divergence Detection (Tertiary):** If the locked choice is no longer valid (e.g., a library was removed from PyPI, an API changed), the system detects this at execution time and reports a `PathLockDivergenceError` with the specific cause.

Partial Locking

Users can lock a subset of decisions:

```
python

class PathLockConfig(BaseModel):
    source_run_id: str
    locked_decisions: list[str]          # Decision IDs to lock, e.g., ["dp-001", "dp-002"]
    # OR
    lock_through: Optional[int]         # Lock all decisions up to sequence N
    # OR
    lock_categories: Optional[list[DecisionCategory]] # Lock all decisions in these categories
```

6.4 Fork Engine

Forking is path locking with a twist: one decision is overridden, everything upstream is locked, and everything downstream runs freely.

```
Original Run:  D1(AAPL) → D2(2yr) → D3(IQR) → D4(FastAPI) → D5(matplotlib)
                                     |
Fork at D3:    D1(AAPL) → D2(2yr) → D3(Isolation Forest) → D4(?) → D5(?)
               ▲ locked   ▲ locked   ▲ overridden          ▲ free   ▲ free
```

The fork engine:

1. Copies decisions D1..D(N-1) as locked
2. Replaces D(N) with the user's override
3. Lets D(N+1)..D(end) run freely
4. Links the forked run to its parent via `parent_run_id` and `fork_point`

6.5 Run Manager

Coordinates the lifecycle of runs. Responsible for:

- Generating run IDs (UUIDv4)
- Managing the run directory structure
- Persisting traces and artifacts
- Supporting run listing, filtering, and comparison

Storage Layout

```

.agent-runs/
├── index.json                # Run metadata index
├── runs/
│   ├── run-abc123/
│   │   ├── trace.json      # Complete DecisionTrace
│   │   ├── trace.jsonl     # Incremental (append-only) trace
│   │   ├── metadata.json   # RunMetadata
│   │   ├── workspace/     # Generated code
│   │   │   ├── main.py
│   │   │   ├── requirements.txt
│   │   │   └── ...
│   │   ├── artifacts/     # Generated outputs
│   │   │   ├── report.html
│   │   │   ├── anomalies.json
│   │   │   └── chart.png
│   │   └── logs/
│   │       ├── llm_calls.jsonl # Raw LLM interactions
│   │       └── execution.log   # Sandbox output
│   └── run-def456/
│       └── ...

```

6.6 LLM Provider Abstraction

```

python

class LLMProvider(Protocol):
    async def complete(
        self,
        messages: list[Message],
        response_model: Optional[type[BaseModel]] = None, # For structured output
        temperature: float = 0.0,
        max_tokens: int = 4096,
    ) -> LLMResponse: ...

class LLMResponse(BaseModel):
    content: str
    structured: Optional[BaseModel] # Parsed if response_model was provided
    usage: TokenUsage
    latency_ms: int
    model: str
    provider: str

```

Implementations: `OpenAIProvider`, `AnthropicProvider`, `OllamaProvider`.

Temperature strategy: Even at temperature=0, LLM outputs are not fully deterministic (batching, quantization, API-side sampling). The system therefore does NOT rely on

temperature for reproducibility — it relies on **decision capture and prompt injection**.

6.7 Code Sandbox

The agent's generated code runs in an isolated environment:

- **Mechanism:** Subprocess with timeout, resource limits, and a virtual environment
- **Isolation:** Separate venv per run (or shared base venv + per-run pip installs)
- **Artifacts:** The sandbox writes outputs to a known directory; the system collects them post-execution
- **Networking:** Allowed (agent needs yfinance access), but rate-limited

```
python

class Sandbox:
    def execute(self, command: str, cwd: Path, timeout: int = 120) -> ExecutionResult:
        """Run a command in the sandbox, capture all output."""
        ...

    def install_dependencies(self, requirements: list[str]) -> ExecutionResult:
        """pip install into the run's venv."""
        ...
```

6.8 Event Bus

For real-time observability and the streaming stretch goal:

```
python

class EventBus:
    def emit(self, event: AgentEvent) -> None: ...
    def subscribe(self, handler: Callable[[AgentEvent], None]) -> None: ...

class AgentEvent(BaseModel):
    event_type: str  # "decision_made", "code_written", "execution_completed"
    run_id: str
    timestamp: datetime
    payload: dict
```

The CLI uses this for progress display. The FastAPI streaming endpoint (stretch) forwards events over WebSocket.

7. Key Data Models (Complete)

```
python
```

— Enums —

```
class RunStatus(str, Enum):  
    RUNNING = "running"  
    SUCCESS = "success"  
    FAILED = "failed"  
    PARTIAL = "partial"           # Crashed mid-run but trace was saved
```

```
class AgentPhase(str, Enum):  
    PLANNING = "planning"  
    CODING = "coding"  
    EXECUTING = "executing"  
    EVALUATING = "evaluating"  
    RECOVERING = "recovering"
```

```
class DecisionCategory(str, Enum):  
    DATA_SELECTION = "data_selection"  
    ALGORITHM_SELECTION = "algorithm_selection"  
    ARCHITECTURE = "architecture"  
    LIBRARY_SELECTION = "library_selection"  
    ERROR_RECOVERY = "error_recovery"  
    OPTIMIZATION = "optimization"  
    PARAMETER_TUNING = "parameter_tuning"
```

— Core Models —

```
class RunMetadata(BaseModel):  
    run_id: str  
    task_description: str  
    llm_provider: str  
    llm_model: str  
    temperature: float  
    started_at: datetime  
    completed_at: Optional[datetime]  
    status: RunStatus  
    parent_run_id: Optional[str]           # For forks  
    fork_point: Optional[str]            # Decision ID where fork occurred  
    lock_config: Optional[PathLockConfig]  
    total_llm_calls: int  
    total_tokens: int  
    total_cost_usd: Optional[float]
```

```
class LLMCallRecord(BaseModel):  
    id: str  
    timestamp: datetime  
    phase: AgentPhase
```



```

prompt_messages: list[dict]          # Full messages array
response: str
structured_output: Optional[dict]
model: str
temperature: float
tokens_in: int
tokens_out: int
latency_ms: int
decision_point_id: Optional[str]     # Which decision this call was for

class RunResult(BaseModel):
    trace: DecisionTrace
    artifacts: list[str]              # Paths to generated files
    error: Optional[str]

class RunComparison(BaseModel):
    """Side-by-side comparison of two runs."""
    run_a_id: str
    run_b_id: str
    decision_alignment: list[DecisionAlignment]
    outcome_diff: OutcomeDiff

class DecisionAlignment(BaseModel):
    """How a decision point aligns between two runs."""
    category: DecisionCategory
    question: str                     # Normalized question
    run_a_choice: Optional[str]
    run_b_choice: Optional[str]
    match: bool

class OutcomeDiff(BaseModel):
    code_line_count_a: int
    code_line_count_b: int
    libraries_a: list[str]
    libraries_b: list[str]
    execution_time_a_ms: int
    execution_time_b_ms: int
    tests_passed_a: Optional[int]
    tests_passed_b: Optional[int]

```

8. API Design

8.1 CLI Interface (P0)

```
deterministic-agent run [OPTIONS] <TASK_FILE_OR_STRING>
  --provider <openai|anthropic|ollama>  LLM provider (default: anthropic)
  --model <model-name>                  Model to use
  --max-iterations <N>                  Max agent loop iterations (default: 10)
  --output-dir <path>                  Override run output directory
  --verbose                             Stream decisions to stdout in real-time

deterministic-agent replay <RUN_ID>
  --verify                             Verify output matches original (diff)

deterministic-agent trace <RUN_ID>
  --format <json|tree|table>            Output format (default: tree)
  --decisions-only                      Omit execution records

deterministic-agent list
  --status <success|failed|partial>     Filter by status
  --limit <N>                           Number of runs to show
  --sort <date|status|decisions>        Sort order

deterministic-agent fork <RUN_ID>
  --decision <DECISION_ID>             Decision point to override
  --choice <VALUE>                     New choice value
  --compare                             Auto-compare with parent after
completion

deterministic-agent compare <RUN_ID_1> <RUN_ID_2>
  --format <table|json|side-by-side>

deterministic-agent analyze
  --runs <RUN_ID,...>                  Specific runs (default: all)
  --output <tree|heatmap|json>         Visualization format
```

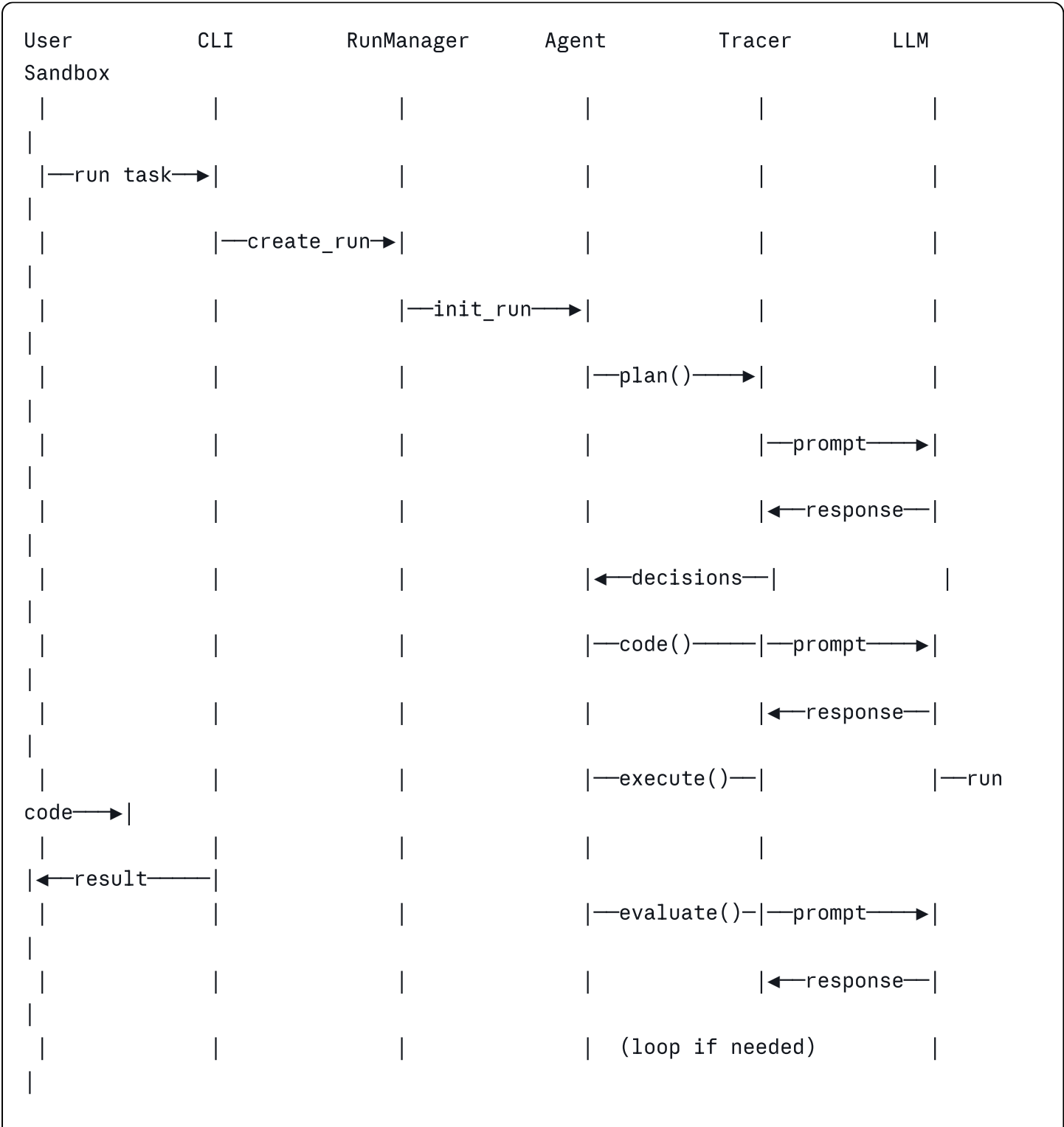
8.2 FastAPI Interface (P2 Stretch)

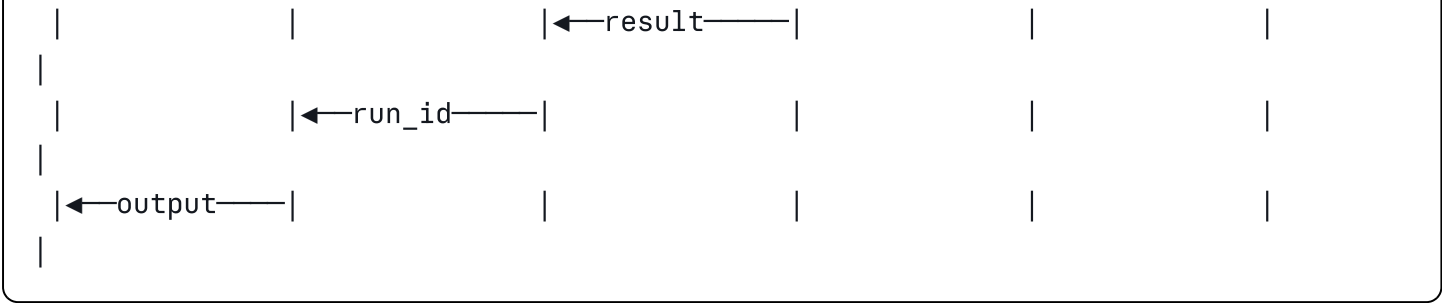
POST	/api/v1/runs	Start a new run
GET	/api/v1/runs	List runs (pagination, filtering)
GET	/api/v1/runs/{run_id}	Get run metadata
GET	/api/v1/runs/{run_id}/trace	Get decision trace
DELETE	/api/v1/runs/{run_id}	Delete a run and its artifacts
POST	/api/v1/runs/{run_id}/replay	Replay a locked path
POST	/api/v1/runs/{run_id}/fork	Fork from a decision point

GET	/api/v1/runs/{run_id}/artifacts	List generated artifacts
GET	/api/v1/runs/{run_id}/artifacts/{name}	Download an artifact
POST	/api/v1/compare	Compare two runs
GET	/api/v1/analysis	Cross-run path analysis
WS	/api/v1/runs/{run_id}/stream	Real-time execution streaming

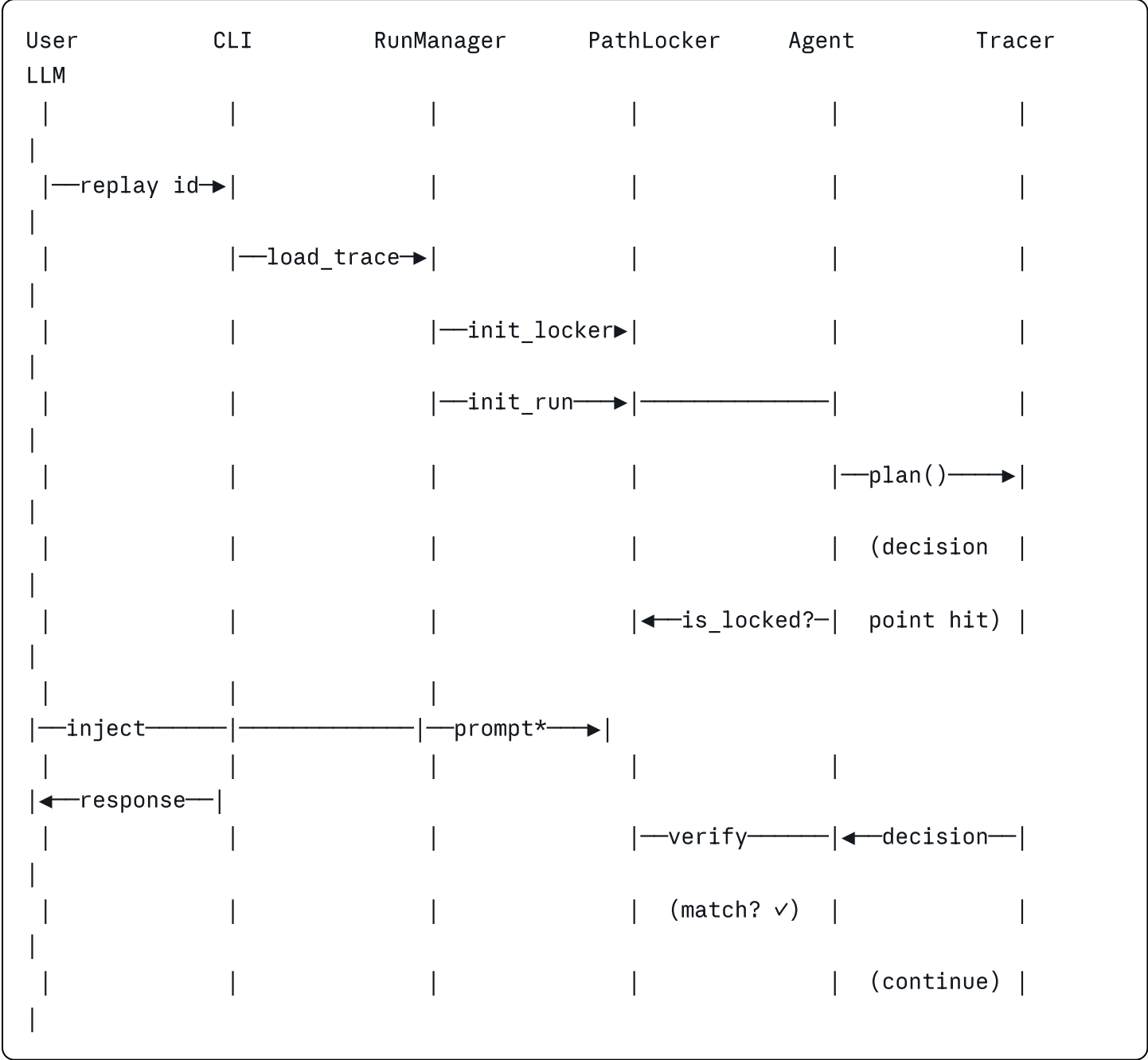
9. Sequence Diagrams

9.1 Normal Run (No Locking)



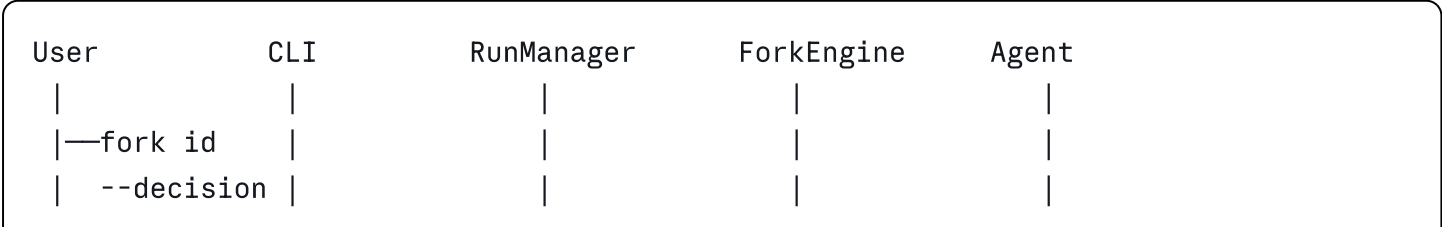


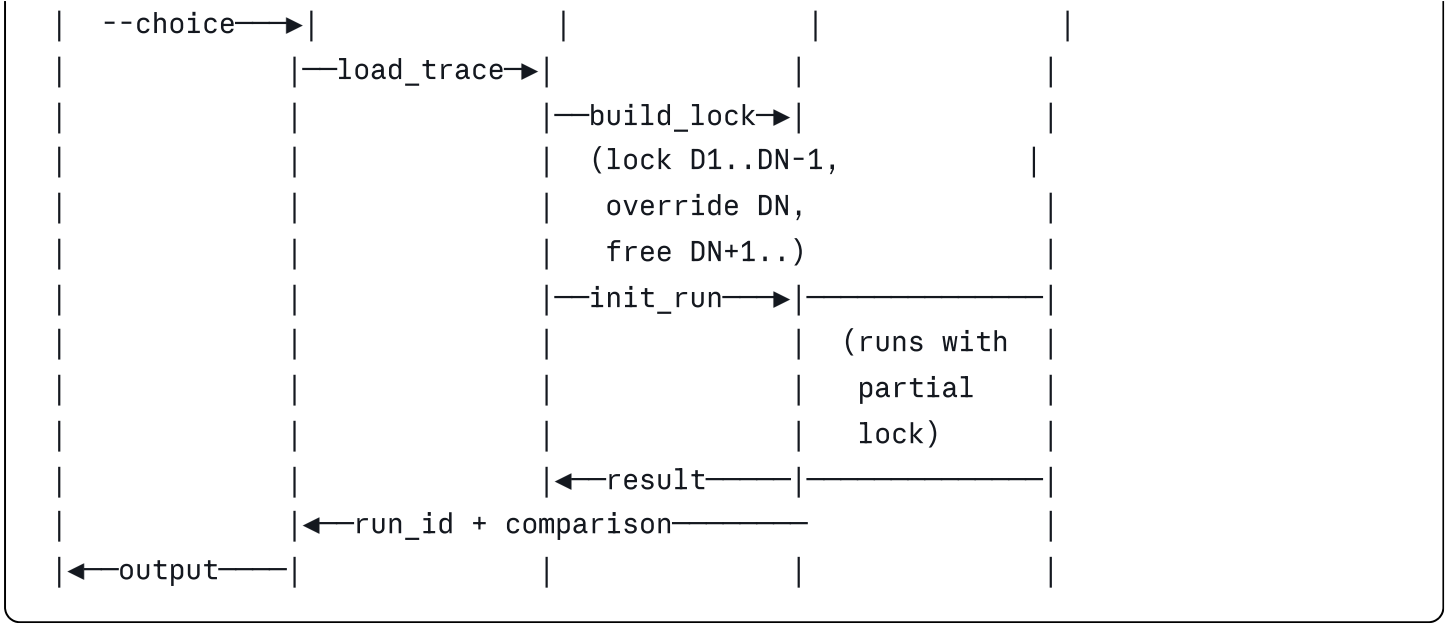
9.2 Locked Replay



*prompt includes: "You MUST choose {locked_value} for {decision_question}."

9.3 Fork & Explore





10. Decision Point Identification Strategy

The reference task has the following known decision points. This taxonomy ensures the agent is prompted to make explicit, traceable choices at each:

10.1 Phase 1: Planning Decisions

#	Decision Point	Typical Alternatives	Impact
1	Ticker selection	AAPL, MSFT, GOOGL, SPY, multi-ticker	Determines data volume, correlation opportunities
2	Timeframe	1yr, 2yr, 5yr, custom range	Affects statistical significance, data size
3	Data fields	Close only, OHLCV, adjusted close, + volume	Determines feature space for anomaly detection
4	Anomaly method	Z-score, IQR, Isolation Forest, rolling std, DBSCAN, autoencoder	Core algorithmic choice — biggest variance driver
5	Anomaly parameters	Threshold (2σ, 3σ), window size, contamination rate	Precision/recall tradeoff

10.2 Phase 2: Architecture Decisions

#	Decision Point	Typical Alternatives	Impact
6	API framework	FastAPI, Flask, raw http.server	Complexity vs. features

#	Decision Point	Typical Alternatives	Impact
7	Report format	HTML, JSON, matplotlib PNG, PDF, structured dataclass	User experience, complexity
8	Project structure	Single file, modular package, src layout	Maintainability
9	Data library	pandas, polars, raw numpy	Performance, API style
10	Visualization library	matplotlib, plotly, seaborn, altair	Interactivity, aesthetic

10.3 Phase 3: Implementation Decisions

#	Decision Point	Typical Alternatives	Impact
11	Error handling strategy	Try/except, validation layer, circuit breaker	Robustness
12	Caching strategy	None, file cache, in-memory	Performance on re-runs
13	API response format	JSON, HTML page, mixed	Consumer experience

10.4 Phase 4: Recovery Decisions (Conditional)

#	Decision Point	Typical Alternatives	Impact
R1	Import error recovery	Install missing package, switch library, inline implementation	Path divergence on failure
R2	Data fetch failure	Retry, fallback ticker, use cached data	Resilience strategy
R3	Test failure recovery	Fix bug, change approach, relax assertion	Quality vs. progress

11. Prompt Engineering Design

11.1 Decision Elicitation Prompt

The key to capturing decisions is forcing the LLM to **externalize its reasoning in structured form** before acting:

You are a coding agent solving a task. Before making any choice, you MUST output a decision record in the following JSON format:

```
{
  "question": "What is the decision being made?",
  "alternatives": [
    {"value": "option_a", "reasoning": "Why this could work"},
    {"value": "option_b", "reasoning": "Why this could work"},
    {"value": "option_c", "reasoning": "Why this could work"}
  ],
  "chosen": "option_b",
  "reasoning": "Why option_b was selected over alternatives",
  "confidence": 0.8,
  "category": "algorithm_selection"
}
```

You must consider at least 2 alternatives for every decision.
Do not skip decision points – even "obvious" choices must be recorded.

11.2 Path Locking Prompt Injection

When a decision is locked, the following is prepended to the decision prompt:

LOCKED DECISION: For the question "{question}", you MUST choose "{locked_value}".
Do not consider alternatives. Output the decision record with the locked value
and set "locked": true. Proceed immediately with this choice.

11.3 Fork Override Prompt

OVERRIDDEN DECISION: For the question "{question}", you previously chose
"{original_value}" but this run is exploring an alternative.
You MUST choose "{override_value}" instead. Adapt your subsequent decisions
accordingly – downstream choices should be the best fit for this new path.

12. Testing Strategy

12.1 Unit Tests

Test	What It Validates
test_decision_point_serialization	DecisionPoint ↔ JSON round-trips correctly
test_trace_completeness	Every agent phase produces at least one decision point

Test	What It Validates
<code>test_path_locker_injection</code>	Locked decisions appear in LLM prompt
<code>test_path_locker_verification</code>	Mismatched decisions raise PathLockViolationError
<code>test_fork_engine_partial_lock</code>	Upstream locked, fork point overridden, downstream free
<code>test_incremental_trace_persistence</code>	Crash mid-run preserves trace up to last decision
<code>test_sandbox_isolation</code>	Agent code runs in subprocess, timeout works
<code>test_decision_category_registry</code>	All categories are registered and parseable

12.2 Integration Tests

Test	What It Validates
<code>test_full_run_produces_artifacts</code>	End-to-end: task → code → execution → report + API
<code>test_locked_replay_matches_original</code>	Replayed run makes identical decisions
<code>test_fork_diverges_at_correct_point</code>	Forked run matches parent upstream, diverges downstream
<code>test_five_runs_show_variance</code>	5 runs on same task produce ≥3 distinct decision paths
<code>test_partial_lock_mixed_behavior</code>	Locked decisions match, free decisions may vary

12.3 Acceptance Criteria for "5 Recorded Runs"

Each of the 5 runs must:

- Complete successfully (produce working code)
- Have a complete decision trace with ≥8 decision points
- Differ from at least 2 other runs in ≥3 decision points
- Pull real data from yfinance
- Produce a functional API endpoint
- Produce an anomaly detection report

13. Key Design Considerations & Trade-offs

13.1 Prompt Injection vs. Post-hoc Filtering for Path Locking

Approach	Pros	Cons
Prompt injection (chosen)	LLM adapts downstream reasoning to locked choice; more natural code; avoids wasted tokens	LLM may ignore the constraint; requires verification layer
Post-hoc filtering	Simpler implementation; guaranteed match	Wastes tokens on reasoning that gets discarded; downstream decisions don't account for the lock; can create incoherent code
Constrained decoding	Strongest guarantee	Only works with local models; brittle for complex structured output

Decision: Prompt injection + verification. The LLM needs to *reason about* the locked decision to produce coherent downstream code. Post-hoc filtering would produce Frankenstein solutions.

13.2 Decision Granularity: Fine vs. Coarse

Approach	Pros	Cons
Fine-grained (every if/else is a decision)	Maximum reproducibility; deep analysis	Trace bloat; many decisions are trivial; high LLM overhead
Coarse-grained (only "big" architectural choices)	Clean traces; fast; meaningful analysis	Misses decision points that cause subtle divergence
Semantic-level (chosen)	Captures decisions humans care about; manageable trace size	Requires careful taxonomy; may miss micro-decisions

Decision: Semantic-level decisions tied to the taxonomy in Section 10. The agent is prompted to make decisions at these known branching points. This is a pragmatic middle ground — we capture ~10-15 decisions per run, which is enough for meaningful analysis without drowning in noise.

13.3 Trace Storage: Filesystem vs. Database

Approach	Pros	Cons
Filesystem (chosen for MVP)	Zero dependencies; portable; git-friendly; human-readable	No indexing; slow cross-run queries at scale
SQLite	Indexed queries; still zero-server; ACID	More complex; harder to inspect manually

Approach	Pros	Cons
PostgreSQL	Full query power; concurrent access; scales	Heavy dependency; deployment complexity

Decision: Filesystem for MVP (JSON files in `.agent-runs/`). Migrate to SQLite if cross-run analysis becomes a bottleneck beyond ~100 runs.

13.4 Sandbox Strategy: Subprocess vs. Container

Approach	Pros	Cons
Subprocess with venv (chosen)	Simple; fast startup; sufficient isolation for this use case	Less isolated than containers; shared filesystem access
Docker container per run	Strong isolation; reproducible environments; clean teardown	Slow startup (~2-5s); Docker dependency; complex networking for yfinance
Firecracker / gVisor	Maximum isolation	Overkill; complex setup

Decision: Subprocess with venv. The agent's code is running locally for a known task — we don't need container-level isolation. The venv provides dependency isolation.

13.5 LLM Temperature: 0.0 vs. Low Positive

Setting	Behavior
<code>temperature=0.0</code>	Most deterministic output per provider, but NOT guaranteed deterministic (batch scheduling, quantization)
<code>temperature=0.7</code>	More creative/varied, but unpredictable — harder to distinguish "decision variance" from "sampling noise"

Decision: Default to `temperature=0.0` for free runs (we want decision-level variance, not token-level noise). For locked runs, temperature is irrelevant since decisions are constrained. This is explicitly NOT how we achieve reproducibility — path locking is.

13.6 Cross-Provider Path Locking Feasibility

Locking a trace from GPT-4 and replaying on Claude is a P2 stretch goal because:

- Decision questions/alternatives may be phrased differently by different models
- The verification layer needs fuzzy matching ("IQR method" vs. "interquartile range")
- Downstream code style will differ even with identical decisions

Mitigation: Normalize decision values to a canonical vocabulary. E.g., `{"IQR", "interquartile_range", "iqr_method"}` all map to `"IQR"`.

14. Failure Modes & Mitigations

Failure Mode	Detection	Mitigation
LLM ignores locked decision	Verification layer detects mismatch	Retry with stronger prompt (up to 3x), then fail with <code>PathLockViolationError</code>
yfinance API down or rate-limited	HTTP error / empty response	Retry with exponential backoff (3 attempts); fall back to cached data if available
Agent enters infinite loop	Iteration counter	Hard cap at <code>MAX_ITERATIONS</code> (default 10); trace preserves progress
Generated code has infinite loop	Sandbox timeout	Kill subprocess after <code>TIMEOUT</code> seconds (default 120)
LLM returns unparseable decision	Pydantic validation failure	Retry with correction prompt; if 3 failures, skip structured capture and log raw response
Disk full during trace write	IOError	Pre-check available space; warn if < 100MB
Forked run makes different decisions upstream	Verification at each locked step	Fail fast with divergence report

15. Phased Delivery Plan

Phase 1: Core (Week 1-2)

- Agent core with plan/code/execute/evaluate loop
- Decision tracer with Pydantic models
- Path locking (prompt injection + verification)
- CLI: `run`, `replay`, `trace`, `list`
- 5 recorded runs demonstrating variance
- Unit + integration tests for trace and locking
- README

Phase 2: Fork & Analysis (Week 3)

- Fork engine with partial locking
- `fork` and `compare` CLI commands
- Cross-run decision tree visualization (terminal-based)
- Variance analysis across runs

Phase 3: API & Streaming (Week 4)

- FastAPI endpoints for all operations
 - WebSocket streaming of agent execution
 - Dashboard UI for visualizing decision trees and comparisons
-

16. Open Questions

1. **Decision normalization:** How aggressively should we normalize decision values across runs? Exact string match vs. semantic similarity?
 2. **Trace versioning:** When the decision taxonomy evolves, how do we handle backward compatibility with old traces?
 3. **Multi-file code generation:** If the agent produces 5 files, how do we trace which decisions led to which files?
 4. **Non-determinism in execution:** If the same code produces slightly different yfinance data (market hours, caching), does that count as a path divergence?
 5. **Cost tracking:** Should we enforce cost caps per run? A 10-iteration run with GPT-4 could cost \$5-10.
-

Appendix A: Reference Task Specification

Task: Pull stock data from yfinance (choose which ticker(s), timeframe, and indicators), detect anomalous trading days using a statistical method of your choosing, generate a summary report with visualizations, and expose the results through a simple API endpoint.

Requirements:

- Must pull real data using yfinance (no mocked datasets)
- Must detect anomalous trading days using a statistical method
- Must generate a summary report with at least one visualization

- Must expose results through an API endpoint (any framework)
- Must handle errors gracefully (missing data, invalid tickers, API issues)

Appendix B: Example Decision Trace (Abbreviated)

json

```
{
  "run_id": "run-a1b2c3",
  "task_description": "Stock anomaly detection pipeline...",
  "status": "success",
  "decisions": [
    {
      "id": "dp-001",
      "sequence_number": 1,
      "category": "data_selection",
      "phase": "planning",
      "question": "Which stock ticker(s) to analyze?",
      "alternatives": [
        {"value": "AAPL", "reasoning": "High liquidity, well-known, clean data"},
        {"value": "TSLA", "reasoning": "High volatility, more anomalies expected"},
        {"value": "SPY", "reasoning": "Index-level, broader market view"}
      ],
      "chosen": "TSLA",
      "reasoning": "TSLA's higher volatility makes anomaly detection more interesting",
      "confidence": 0.75,
      "locked": false
    },
    {
      "id": "dp-002",
      "sequence_number": 2,
      "category": "data_selection",
      "phase": "planning",
      "question": "What timeframe for historical data?",
      "alternatives": [
        {"value": "1y", "reasoning": "Recent, relevant, fast to fetch"},
        {"value": "2y", "reasoning": "More data points, better statistical significance"},
        {"value": "5y", "reasoning": "Long-term patterns, but includes COVID anomaly"}
      ],
      "chosen": "2y",
      "reasoning": "2 years gives ~500 trading days – enough for statistical methods",
      "confidence": 0.8,
      "locked": false
    },
    {
      "id": "dp-003",
      "sequence_number": 3,
      "category": "algorithm_selection",
      "phase": "planning",
      "question": "Which anomaly detection method?",
      "alternatives": [
        {"value": "z_score", "reasoning": "Simple, interpretable, fast"},
        {"value": "iqr", "reasoning": "Robust to outliers, non-parametric"},

```

```
    {"value": "isolation_forest", "reasoning": "Handles multivariate data, no di
  ],
  "chosen": "isolation_forest",
  "reasoning": "With OHLCV data, multivariate anomaly detection captures volume
  "confidence": 0.7,
  "locked": false
}
]
```

Appendix C: Decision Variance Classification

C.1 Variance Tiers

Not all decisions are equal. Some fundamentally change the output; others are cosmetic. This classification drives where we invest tracing effort, where we pin defaults, and where caching is effective.

Tier	Variance Score	Meaning	Tracing Strategy
HIGH	7.5 - 10	Output is structurally different (different anomalies found, different data, different analytical conclusions)	Full trace with alternatives, fork-and-explore enabled, no pinning
MEDIUM	5 - 7	Output shape or presentation changes, but analytical substance is similar	Trace the decision, but fork-and-explore is lower priority
LOW	2 - 3.5	Code changes but output is functionally identical	Trace for completeness, but pin to a default in cost-sensitive mode
NO-OP	1 - 1.5	Purely cosmetic/structural code difference, zero output impact	Pin to a default. Do not waste tokens exploring alternatives

C.2 Full Classification Table

#	Decision Point	Alternatives	Variance	Tier	Rationale
1	Anomaly detection method	z-score, IQR, Isolation Forest, rolling std, DBSCAN	9.5	HIGH	Different methods find fundamentally different anomalies. Z-score catches univariate spikes; Isolation Forest catches multivariate clusters. The set of

#	Decision Point	Alternatives	Variance	Tier	Rationale
					"anomalous days" can differ by 50%+ between methods.
2	Ticker selection	AAPL, TSLA, MSFT, SPY, multi-ticker	9.0	HIGH	Different stocks have different volatility profiles, sector exposure, and anomaly signatures. TSLA daily swings make z-score useless; SPY is so stable that only extreme events register.
3	Timeframe	1yr, 2yr, 5yr, custom	8.0	HIGH	1yr = ~250 data points (weak statistics), 5yr includes COVID (massive outlier contamination). Statistical significance of detected anomalies changes entirely.
4	Anomaly parameters	threshold ($2\sigma/3\sigma$), contamination rate, window size	7.5	HIGH	2σ threshold catches ~5% of days; 3σ catches ~0.3%. This is a 15x difference in output size. Window size for rolling methods changes whether "anomalous" means "unusual today" vs "unusual this quarter."
5	Data fields	Close only, OHLCV, adjusted close	7.0	MEDIUM	OHLCV enables multivariate detection (volume spikes + price drops); close-only limits to univariate. Changes the feature space but if the method is univariate anyway, the extra fields are unused.
6	Report format	HTML, JSON, matplotlib PNG, PDF	6.5	MEDIUM	Changes what the user sees but not what was computed. HTML report with interactive charts vs a static PNG are different UX but same data. JSON output is the most divergent (no visualization at all).
7	Error recovery strategy	Fix code, change approach, fallback	6.0	MEDIUM	Only matters when errors occur. If no errors, this decision never fires. When it does fire, "change approach" (e.g., switch from Isolation Forest to z-score because sklearn import failed) can cascade into HIGH variance. Conditional impact.
8	Visualization library	matplotlib, plotly, seaborn, altair	5.0	MEDIUM	Plotly produces interactive HTML charts; matplotlib produces static PNGs. Different user experience but identical

#	Decision Point	Alternatives	Variance	Tier	Rationale
					underlying data. Seaborn adds aesthetic defaults. The analytical content is unchanged.
9	Project structure	Single file, modular, src layout	3.5	LOW	Zero impact on output. <code>main.py</code> with 200 lines vs <code>src/detector.py</code> + <code>src/api.py</code> + <code>src/report.py</code> produce identical behavior. Only matters for maintainability, which isn't measured in this system.
10	Data library	pandas, polars, raw numpy	2.0	LOW	<code>df.groupby().mean()</code> in pandas and <code>df.group_by().mean()</code> in polars produce the same numbers. Performance differs (polars is faster) but output is byte-identical. Pin to pandas for ecosystem compatibility.
11	API framework	FastAPI, Flask, raw http.server	1.5	NO-OP	Same endpoints, same JSON responses, same status codes. The difference is 3 import lines and a decorator syntax. <code>@app.get("/anomalies")</code> vs <code>@app.route("/anomalies")</code> . Pin to FastAPI.
12	Caching strategy	None, file cache, in-memory	1.0	NO-OP	Invisible to the end user. Only affects re-run performance. Pin to "none" for simplicity in agent-generated code.
13	Error handling pattern	try/except, validation layer, circuit breaker	1.0	NO-OP	Same behavior for the user. Try/except catches the same errors as a validation layer; the code structure differs but the API returns the same error responses. Pin to try/except.

C.3 Recommended Pinning Strategy

For cost-sensitive runs (or when the user cares about analytical variance, not code-style variance), the system should auto-pin NO-OP and LOW tier decisions:

```
python
```

```

DEFAULT_PINS = {
    "api_framework": "fastapi",
    "data_library": "pandas",
    "caching_strategy": "none",
    "error_handling_pattern": "try_except",
    "project_structure": "single_file",
}

```

This reduces the effective decision space from ~13 decisions to ~8, saving approximately 30-40% of LLM tokens per run while preserving all meaningful variance.

C.4 Variance-Aware Decision Point Registry

```

python

class DecisionPointConfig(BaseModel):
    category: DecisionCategory
    variance_tier: VarianceTier          # HIGH, MEDIUM, LOW, NOOP
    variance_score: float                 # 0-10
    default_pin: Optional[str]           # Auto-pin value for cost-sensitive mode
    fork_enabled: bool                   # Whether fork-and-explore is meaningful
    alternatives_budget: int              # Max alternatives to consider (saves tokens)

DECISION_REGISTRY = {
    "anomaly_method": DecisionPointConfig(
        category=DecisionCategory.ALGORITHM_SELECTION,
        variance_tier=VarianceTier.HIGH,
        variance_score=9.5,
        default_pin=None,                 # Never pin – this is the whole point
        fork_enabled=True,
        alternatives_budget=5,            # Explore many options
    ),
    "api_framework": DecisionPointConfig(
        category=DecisionCategory.ARCHITECTURE,
        variance_tier=VarianceTier.NOOP,
        variance_score=1.5,
        default_pin="fastapi",           # Always pin
        fork_enabled=False,               # Forking here is pointless
        alternatives_budget=1,           # Don't even enumerate alternatives
    ),
    # ...
}

```

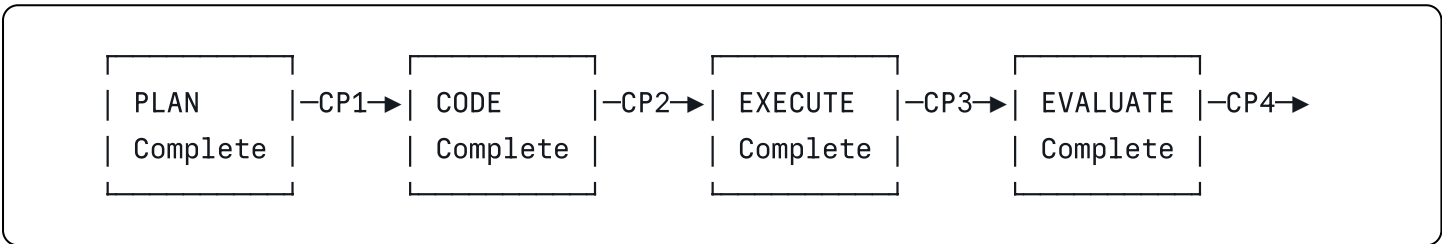
Appendix D: Checkpoint Caching System

D.1 Motivation

A full agent run involves 5-15 LLM calls, each costing tokens and time. In a locked replay, many of these calls are redundant — the LLM is being asked to make a decision we already know the answer to. Checkpoint caching allows us to **skip LLM calls entirely** for segments where the decision path is identical to a cached run.

D.2 Checkpoint Boundaries

The agent has four natural checkpoint boundaries, each representing a stable state that can be serialized and restored:



Checkpoint	What's Cached	Size	Cache Hit Criteria
CP1: Post-Plan	Decision trace (decisions 1-N), plan document, chosen approach	~50KB	Same task hash + same locked decisions through planning phase
CP2: Post-Code	All of CP1 + generated code files, requirements.txt, file hashes	~200KB	CP1 match + same code-phase decisions (library, structure, patterns)
CP3: Post-Execute	All of CP2 + execution results (stdout, stderr, exit code, artifacts)	~2MB	CP2 match + same execution environment (Python version, package versions)
CP4: Post-Evaluate	All of CP3 + evaluation assessment, next-step decision	~2.1MB	CP3 match + same evaluation criteria

D.3 Cache Hit Criteria

The core question: **when can we skip the LLM call and use cached state?**

```
python
```

```

class CheckpointKey(BaseModel):
    """Deterministic key for cache lookup."""
    task_hash: str # SHA-256 of normalized task description
    decisions_hash: str # SHA-256 of ordered (question, chosen) pair
    phase: AgentPhase
    iteration: int # Which loop iteration (0-indexed)

    # Environmental factors that affect execution (CP3 only)
    python_version: Optional[str]
    package_versions: Optional[dict[str, str]] # {package: version}

class Checkpoint(BaseModel):
    key: CheckpointKey
    state: dict # Serialized agent state at this point
    decisions_so_far: list[DecisionPoint]
    code_snapshot: Optional[str] # File contents at this point
    execution_result: Optional[ExecutionRecord]
    created_at: datetime
    source_run_id: str # Which run created this checkpoint

```

D.4 Cache Match Logic

```
python
```

```

class CheckpointCache:
    def lookup(self, current_key: CheckpointKey) -> Optional[Checkpoint]:
        """
        Match criteria (all must hold):
        1. task_hash matches (same task description)
        2. decisions_hash matches (same choices made up to this point)
        3. phase matches
        4. iteration matches
        5. For CP3+: python_version matches and package_versions are compatible

        "Compatible" for packages: exact version match for core packages
        (pandas, numpy, sklearn, yfinance), minor-version tolerance for others.
        """
        ...

    def should_use_cache(self, checkpoint: Checkpoint, lock_config: PathLockConfig)
        """
        Additional validation before using a cached checkpoint:
        - If the next decision is locked AND matches the cached decision → USE CACHE
        - If the next decision is locked but differs from cache → SKIP CACHE (re-run)
        - If the next decision is free → SKIP CACHE (must let LLM decide freely)
        - If checkpoint is older than MAX_CACHE_AGE → SKIP CACHE (yfinance data changes)
        """
        ...

```

D.5 Cache Invalidation Strategy

Invalidation Trigger	What Gets Invalidated	Rationale
yfinance data changes (new trading day)	CP3, CP4 for any checkpoint using live data	Execution results will differ
LLM model version changes	All checkpoints	Same prompt may produce different structured output
Package version bump (major)	CP2, CP3, CP4	Code behavior may change
Task description edit (even whitespace)	All checkpoints	Task hash changes
Decision override at fork point	All checkpoints downstream of fork	Cascade invalidation
Manual agent cache clear	Everything	User-initiated full clear

D.6 Token Savings Estimate

For a locked replay of a 10-decision run:

Scenario	LLM Calls	Est. Tokens	Est. Cost (Claude Sonnet)
No caching (full replay)	10-15	~50K	~\$0.45
Checkpoint caching (all decisions locked, cache hit)	0-2	~5K	~\$0.05
Partial lock (5 locked, 5 free)	5-8	~25K	~\$0.23
Free run (no locking, no caching)	10-15	~50K	~\$0.45

Checkpoint caching provides ~**90% token reduction** on full replays and ~**50% reduction** on partial replays. The savings come from skipping planning-phase LLM calls entirely when all planning decisions are locked and cached.

Appendix E: Tiered Cache Architecture

E.1 Three-Tier Cache Design

The system uses a tiered cache to balance speed, capacity, and persistence:

L1: In-Memory (per-process)
TTL: session lifetime Size: 50MB
Use: hot-path lookups during active run
Contents: current run's checkpoints, recent decision traces, LLM response cache
L2: Redis (shared across processes)
TTL: 24h default, configurable Size: 500MB
Use: cross-run caching, concurrent run sharing
Contents: checkpoint snapshots, decision traces, yfinance data cache, cost tracking counters
L3: Filesystem (persistent)
TTL: indefinite Size: unbounded
Use: permanent trace storage, artifact archive
Contents: complete traces, generated code, reports, comparison data

E.2 What Goes Where

Data Type	L1 (Memory)	L2 (Redis)	L3 (Filesystem)
Active run's decision trace (in progress)	✓ Primary	✓ Backup (write-behind)	✓ Append-only JSONL
Completed decision traces	✗ Evicted on run end	✓ 24h TTL	✓ Permanent
Checkpoint snapshots	✓ Current run only	✓ Cross-run sharing	✗ Too large for permanent storage
LLM response cache (exact prompt → response)	✓ Session-scoped	✓ 1h TTL, model-versioned	✗
yfinance data cache	✗	✓ Market-hours-aware TTL	✗
Cost tracking counters	✓ Atomic in-process	✓ Atomic cross-process	✗
Generated code artifacts	✗	✗	✓ Per-run directory
Run comparison results	✗	✓ 1h TTL	✓ On-demand

E.3 Implementation

python


```

from abc import ABC, abstractmethod
from typing import Optional, Any
import hashlib, json, time

class CacheLayer(ABC):
    @abstractmethod
    async def get(self, key: str) -> Optional[Any]: ...
    @abstractmethod
    async def set(self, key: str, value: Any, ttl_seconds: Optional[int] = None) ->
    @abstractmethod
    async def delete(self, key: str) -> None: ...
    @abstractmethod
    async def exists(self, key: str) -> bool: ...

class InMemoryCache(CacheLayer):
    """L1: Process-local, LRU eviction, no serialization overhead."""
    def __init__(self, max_size_mb: int = 50):
        self._store: dict[str, tuple[Any, float]] = {} # key → (value, expires_at)
        self._max_size_mb = max_size_mb

class RedisCache(CacheLayer):
    """L2: Shared across processes, supports atomic operations."""
    def __init__(self, redis_url: str = "redis://localhost:6379", prefix: str = "dag"):
        self._prefix = prefix
        # Connection pool with retry logic
        ...

    async def increment_cost(self, run_id: str, amount_usd: float) -> float:
        """Atomic cost counter increment. Returns new total."""
        key = f"{self._prefix}cost:{run_id}"
        return await self._redis.incrbyfloat(key, amount_usd)

    async def get_cost(self, run_id: str) -> float:
        """Get current accumulated cost for a run."""
        key = f"{self._prefix}cost:{run_id}"
        val = await self._redis.get(key)
        return float(val) if val else 0.0

class FilesystemCache(CacheLayer):
    """L3: Persistent, human-readable, git-friendly."""
    def __init__(self, base_dir: str = ".agent-runs"):
        self._base_dir = Path(base_dir)

class TieredCache:
    """Orchestrates L1 → L2 → L3 lookups with write-through."""

```

```

def __init__(self, l1: InMemoryCache, l2: RedisCache, l3: FileSystemCache):
    self.l1 = l1
    self.l2 = l2
    self.l3 = l3

async def get(self, key: str, tier: str = "all") -> Optional[Any]:
    """
    Lookup order: L1 → L2 → L3.
    On L2/L3 hit, backfill higher tiers.
    """
    if tier in ("all", "l1"):
        result = await self.l1.get(key)
        if result is not None:
            return result

    if tier in ("all", "l2"):
        result = await self.l2.get(key)
        if result is not None:
            await self.l1.set(key, result) # Backfill L1
            return result

    if tier in ("all", "l3"):
        result = await self.l3.get(key)
        if result is not None:
            await self.l2.set(key, result, ttl_seconds=3600) # Backfill L2
            await self.l1.set(key, result) # Backfill L1
            return result

    return None

async def set(self, key: str, value: Any, persist: bool = False, ttl_seconds: Op
    """
    Write-through: always writes to L1 + L2.
    Set persist=True to also write to L3 (filesystem).
    """
    await self.l1.set(key, value, ttl_seconds)
    await self.l2.set(key, value, ttl_seconds)
    if persist:
        await self.l3.set(key, value)

```

E.4 Redis Key Schema

```

dagent:checkpoint:{task_hash}:{decisions_hash}:{phase}:{iteration} → Checkpoint
JSON
dagent:trace:{run_id} →
DecisionTrace JSON

```

dagent:cost:{run_id}	→ float
(accumulated USD)	
dagent:cost:estimate:{task_hash}	→
CostEstimate JSON	
dagent:yfinance:{ticker}:{timeframe}:{date}	→ DataFrame
JSON	
dagent:llm_cache:{prompt_hash}:{model}:{temperature}	→ LLM response
string	
dagent:lock:{run_id}	→ distributed
lock (SETNX)	

E.5 yfinance Data Caching (Special Case)

yfinance data is the most important thing to cache because it's slow (~2-5s per call), rate-limited, and deterministic within a trading day.

```
python

class YFinanceCache:
    """Market-hours-aware caching for stock data."""

    def __init__(self, redis: RedisCache):
        self._redis = redis

    def _ttl_for_market_data(self) -> int:
        """
        - During market hours (9:30 AM - 4:00 PM ET, Mon-Fri): 5 minutes
        - After hours / weekends: until next market open
        - Historical data (timeframe > 1 day): 24 hours
        """
        ...

    async def get_stock_data(self, ticker: str, period: str, interval: str = "1d") -
        key = f"dagent:yfinance:{ticker}:{period}:{interval}:{date.today()}"
        cached = await self._redis.get(key)
        if cached:
            return pd.read_json(cached)

        # Cache miss - fetch from yfinance
        df = yf.download(ticker, period=period, interval=interval)
        ttl = self._ttl_for_market_data()
        await self._redis.set(key, df.to_json(), ttl_seconds=ttl)
        return df
```

E.6 Redis as Optional Dependency

Redis is NOT required for the system to function. The system degrades gracefully:

Redis Status	Behavior
Available	Full tiered caching, cross-run checkpoint sharing, atomic cost tracking
Unavailable	L1 (memory) + L3 (filesystem) only. Cost tracking uses in-memory counters per process. Checkpoint sharing limited to current process.
Connection lost mid-run	Automatic fallback to L1 + L3. Warning logged. No data loss (L3 is always written).

```
python

class CacheFactory:
    @staticmethod
    def create(config: CacheConfig) -> TieredCache:
        l1 = InMemoryCache(max_size_mb=config.l1_size_mb)
        l3 = FilesystemCache(base_dir=config.runs_dir)

        if config.redis_url:
            try:
                l2 = RedisCache(redis_url=config.redis_url)
                l2.ping() # Verify connection
            except ConnectionError:
                logger.warning("Redis unavailable, falling back to memory-only L2")
                l2 = InMemoryCache(max_size_mb=config.l2_fallback_size_mb)
        else:
            l2 = InMemoryCache(max_size_mb=config.l2_fallback_size_mb)

        return TieredCache(l1, l2, l3)
```

Appendix F: Cost Control System

F.1 Motivation

A single agent run can involve 10-15 LLM calls. With GPT-4o at ~\$5/1M input tokens and Claude Sonnet at ~\$3/1M input, an unconstrained run can cost \$0.30-\$2.00. Multi-iteration runs with retries can exceed \$5.00. Users need visibility and control.

F.2 Cost Model

```
python
```

```

# Cost per 1M tokens (as of March 2026, approximate)
COST_TABLE = {
    "gpt-4o": {"input": 2.50, "output": 10.00},
    "gpt-4o-mini": {"input": 0.15, "output": 0.60},
    "claude-sonnet-4": {"input": 3.00, "output": 15.00},
    "claude-haiku-4": {"input": 0.80, "output": 4.00},
    "ollama/*": {"input": 0.00, "output": 0.00}, # Local, free
}

class CostTracker:
    def __init__(self, max_cost_usd: float = 2.00, cache: TieredCache = None):
        self.max_cost_usd = max_cost_usd
        self.accumulated_usd = 0.0
        self._cache = cache

    def record_llm_call(self, call: LLMCallRecord) -> None:
        """Record cost and check against budget."""
        cost = self._calculate_cost(call)
        self.accumulated_usd += cost

        # Atomic increment in Redis for cross-process accuracy
        if self._cache:
            self._cache.l2.increment_cost(self.run_id, cost)

        if self.accumulated_usd >= self.max_cost_usd:
            raise CostLimitExceededError(
                f"Run cost ${self.accumulated_usd:.2f} exceeds limit ${self.max_cost_usd:.2f}. "
                f"Run halted at decision {self.current_decision_id}. "
                f"Partial trace saved. Use --max-cost to increase limit."
            )

    def _calculate_cost(self, call: LLMCallRecord) -> float:
        rates = COST_TABLE.get(call.model, COST_TABLE.get("claude-sonnet-4"))
        input_cost = (call.tokens_in / 1_000_000) * rates["input"]
        output_cost = (call.tokens_out / 1_000_000) * rates["output"]
        return input_cost + output_cost

```

F.3 Upfront Cost Estimator

Before a run starts, the estimator predicts cost based on task complexity and configuration:

```
python
```

```

class CostEstimator:
    """Predicts run cost before execution. Shown to user in CLI for confirmation."""

    # Empirical averages from observed runs
    TOKENS_PER_DECISION = {
        VarianceTier.HIGH: {"input": 3000, "output": 800},      # Full enumeration + r
        VarianceTier.MEDIUM: {"input": 2000, "output": 500},   # Moderate exploration
        VarianceTier.LOW: {"input": 500, "output": 200},        # Pinned, minimal prom
        VarianceTier.NOOP: {"input": 200, "output": 100},        # Auto-pinned, near-ze
    }

    TOKENS_PER_CODE_ITERATION = {"input": 4000, "output": 2000}
    TOKENS_PER_ERROR_RECOVERY = {"input": 3000, "output": 1500}

    def estimate(self,
                  model: str,
                  num_decisions: dict[VarianceTier, int],
                  estimated_iterations: int = 2,
                  estimated_errors: int = 1,
                  lock_config: Optional[PathLockConfig] = None) -> CostEstimate:
        """
        Estimate total cost for a run.

        If lock_config is provided, locked decisions are priced at NOOP rate
        (or zero if checkpoint cache is expected to hit).
        """
        total_input = 0
        total_output = 0

        for tier, count in num_decisions.items():
            if lock_config and self._is_locked(tier, lock_config):
                # Locked decisions use minimal tokens (or zero with cache hit)
                tokens = self.TOKENS_PER_DECISION[VarianceTier.NOOP]
            else:
                tokens = self.TOKENS_PER_DECISION[tier]
            total_input += tokens["input"] * count
            total_output += tokens["output"] * count

        # Code generation iterations
        total_input += self.TOKENS_PER_CODE_ITERATION["input"] * estimated_iterations
        total_output += self.TOKENS_PER_CODE_ITERATION["output"] * estimated_iterations

        # Error recovery (probabilistic)
        total_input += self.TOKENS_PER_ERROR_RECOVERY["input"] * estimated_errors
        total_output += self.TOKENS_PER_ERROR_RECOVERY["output"] * estimated_errors

```

```

rates = COST_TABLE.get(model)
estimated_cost = (
    (total_input / 1_000_000) * rates["input"] +
    (total_output / 1_000_000) * rates["output"]
)

return CostEstimate(
    model=model,
    estimated_tokens_in=total_input,
    estimated_tokens_out=total_output,
    estimated_cost_usd=estimated_cost,
    confidence_interval=(estimated_cost * 0.6, estimated_cost * 1.8), # 60%
    breakdown={
        "decisions": self._decision_breakdown(num_decisions, lock_config),
        "code_iterations": estimated_iterations,
        "error_recovery": estimated_errors,
    }
)

class CostEstimate(BaseModel):
    model: str
    estimated_tokens_in: int
    estimated_tokens_out: int
    estimated_cost_usd: float
    confidence_interval: tuple[float, float] # (low, high) in USD
    breakdown: dict

```

F.4 CLI Integration

```
$ deterministic-agent run "Stock anomaly detection pipeline..." --model claude-sonnet-4
```

```

|                                     Cost Estimate
|
| Model:      claude-sonnet-4
| Decisions:  8 free (4 HIGH, 2 MEDIUM, 2 pinned)
| Est. tokens: ~45K in / ~12K out
| Est. cost:   $0.31 (range: $0.19 - $0.56)
| Max budget:  $2.00 (override with --max-cost)
|
| Breakdown:
|   Planning decisions:  $0.08
|   Code generation:     $0.15
|   Error recovery:      $0.08
|

```

Proceed? [Y/n]:

If the user says no, the run doesn't start. If the estimate exceeds `--max-cost`, the CLI warns before asking:

```
$ deterministic-agent run "..." --model gpt-4o --max-cost 0.50
```

▲ Estimated cost \$0.78 exceeds your budget of \$0.50.

Suggestions:

- Use `--model claude-haiku-4` (est. \$0.12)
- Use `--pin-low` to auto-pin LOW/NOOP decisions (est. \$0.52)
- Use `--max-cost 1.00` to increase budget

Proceed anyway? [y/N]:

F.5 Runtime Cost Enforcement

During execution, the cost tracker checks after every LLM call:

```
python
```

```
class AgentRunner:
    def run(self, task: str, config: RunConfig) -> RunResult:
        cost_tracker = CostTracker(
            max_cost_usd=config.max_cost_usd,  # Default: $2.00
            cache=self.cache,
        )

        for iteration in range(config.max_iterations):
            try:
                # Every LLM call goes through cost_tracker
                response = await self.llm.complete(messages, cost_tracker=cost_tracker)
            except CostLimitExceededError as e:
                # Save partial trace before halting
                tracer.finalize(status=RunStatus.PARTIAL, error=str(e))
                return RunResult(
                    trace=tracer.trace,
                    error=f"Budget exceeded: {e}",
                    partial=True,
                )
```

F.6 Updated CLI Commands

```
deterministic-agent run <TASK> [OPTIONS]
    --max-cost <USD>                Maximum cost per run (default: $2.00)
```



```
--estimate-only          Show cost estimate without running
--pin-low                Auto-pin LOW and NOOP variance decisions
--model <model>          LLM model (affects cost estimate)
--no-confirm             Skip cost confirmation prompt
```

```
deterministic-agent estimate <TASK> [OPTIONS]
  --model <model>          Model to estimate for
  --compare-models         Show estimates for all available models
  --with-cache             Factor in expected cache hits
```

F.7 Cost-Per-Decision Visibility

The decision trace includes per-decision cost attribution:

```
python

class DecisionPoint(BaseModel):
    # ... existing fields ...
    cost_usd: float           # Cost of the LLM call(s) for this decision
    tokens_in: int            # Input tokens for this decision
    tokens_out: int           # Output tokens for this decision
    cache_hit: bool           # Was this decision served from cache?
```

This enables post-run analysis: "Which decisions are most expensive? Are we spending \$0.50 on a NO-OP decision because the prompt is too verbose?"

F.8 Default Cost Limits

Mode	Default Max Cost	Rationale
Free run	\$2.00	Covers ~15 LLM calls with buffer for retries
Locked replay (full)	\$0.50	Most decisions served from cache
Locked replay (partial)	\$1.00	Mix of cached and free decisions
Fork-and-explore	\$1.50	Upstream cached, downstream free
<code>--estimate-only</code>	\$0.00	No LLM calls, just calculation